ESTÁCIO

Trabalho Prático

Interação com sensores de smartphones e wearables

DGT2816

App Doma Assistente para Wear OS

Aluno: Gabriel Cosso

Repositório: https://github.com/biel-cosso/doma-assistente-wearos

Data: outubro de 2026

1. Objetivo

O trabalho pede um aplicativo Wear OS para a empresa Doma, que quer ajudar funcionários com necessidades especiais usando áudio: ler mensagens e notificações em voz alta, responder a comandos de voz e dar alertas de segurança. Além do app, os objetivos eram instalar o Android Studio e o emulador, criar o projeto Wear OS, rodar no emulador e tirar prints.

O app que eu fiz se chama **Doma Assistente**. Ele mostra uma lista de avisos (mensagens, lembretes, dicas de treinamento e alertas de segurança) e lê cada um em voz alta. Os alertas tocam um bipe e vibram antes de falar.

2. Microatividades

Microatividade 1: instalação do Android Studio

Instalei o Android Studio 2026.2.1 no Windows 11. Junto vieram o Android SDK, o emulador e a imagem do relógio. O computador não tinha a virtualização ligada para o emulador, então também instalei o Android Emulator Hypervisor Driver, senão o emulador ficaria muito lento.

Microatividade 2: criando o projeto

Criei o projeto `DomaAssistente` (pacote `com.example.domaassistente`) para Wear OS, com Minimum SDK API 30 (Android 11), como o roteiro pede. Usei Kotlin, que é a linguagem dos exemplos de código do roteiro.

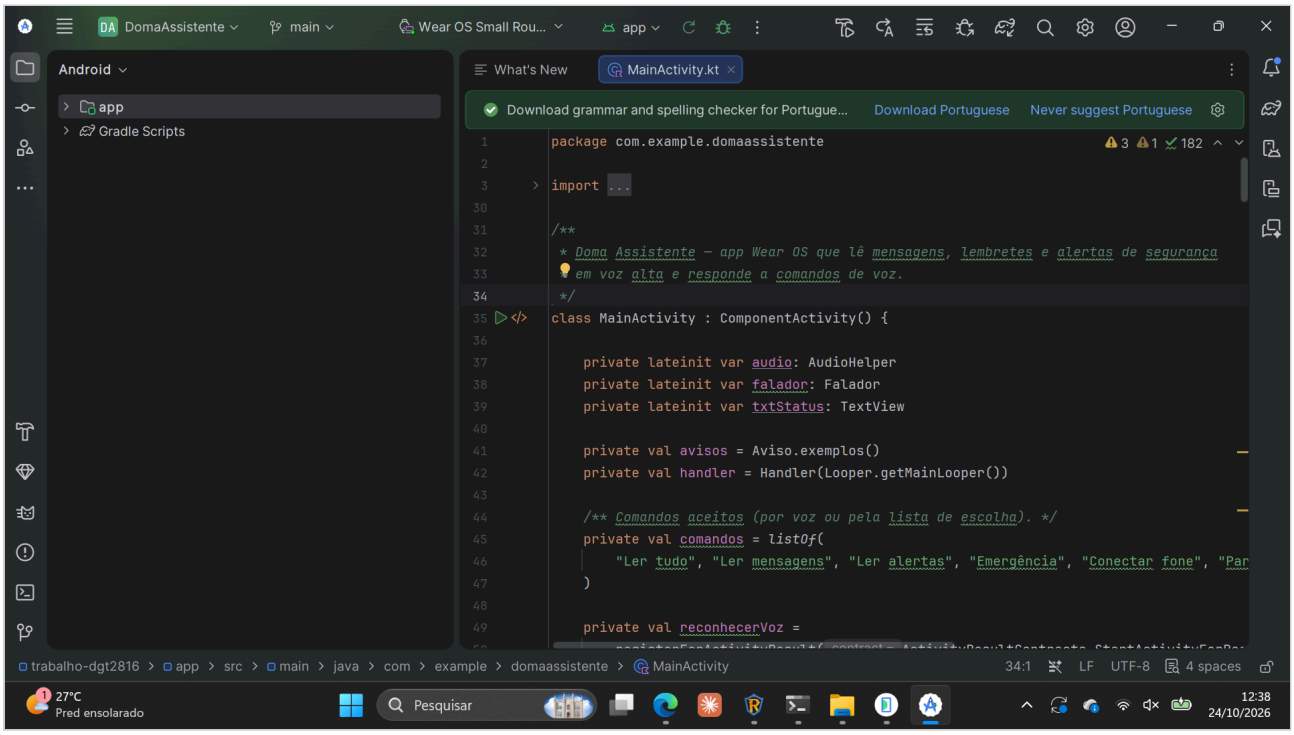


Figura 1 - Projeto aberto no Android Studio, com o emulador Wear OS Small Round selecionado em cima.

Microatividade 3: arquivos de lógica e configuração

Fiz a `MainActivity` com o layout `activity_main`, que tem a `ListView` com os avisos e o `Button` de comando de voz. No `AndroidManifest.xml` coloquei as permissões `BODY_SENSORS` e `WAKE_LOCK` do roteiro, a `VIBRATE` (para os alertas) e o `intent-filter` com MAIN e LAUNCHER. Depois sincronizei o Gradle.

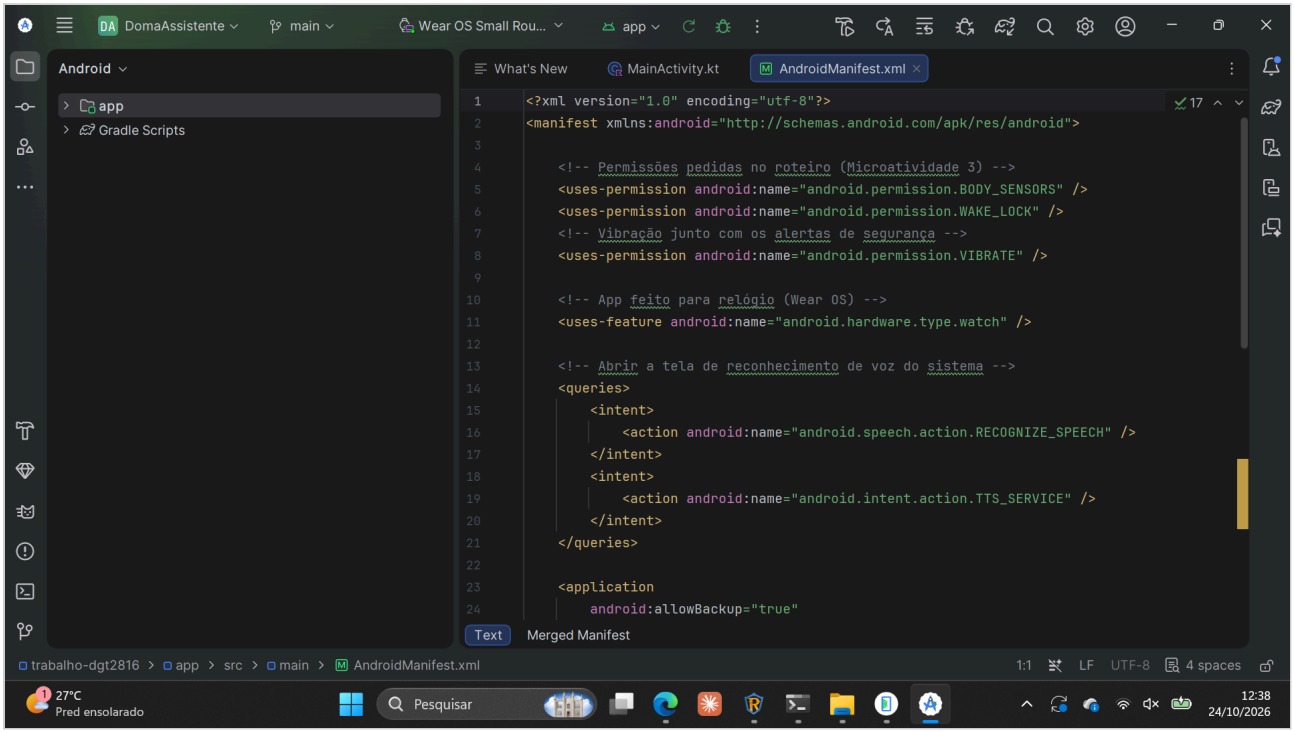


Figura 2 - AndroidManifest.xml.

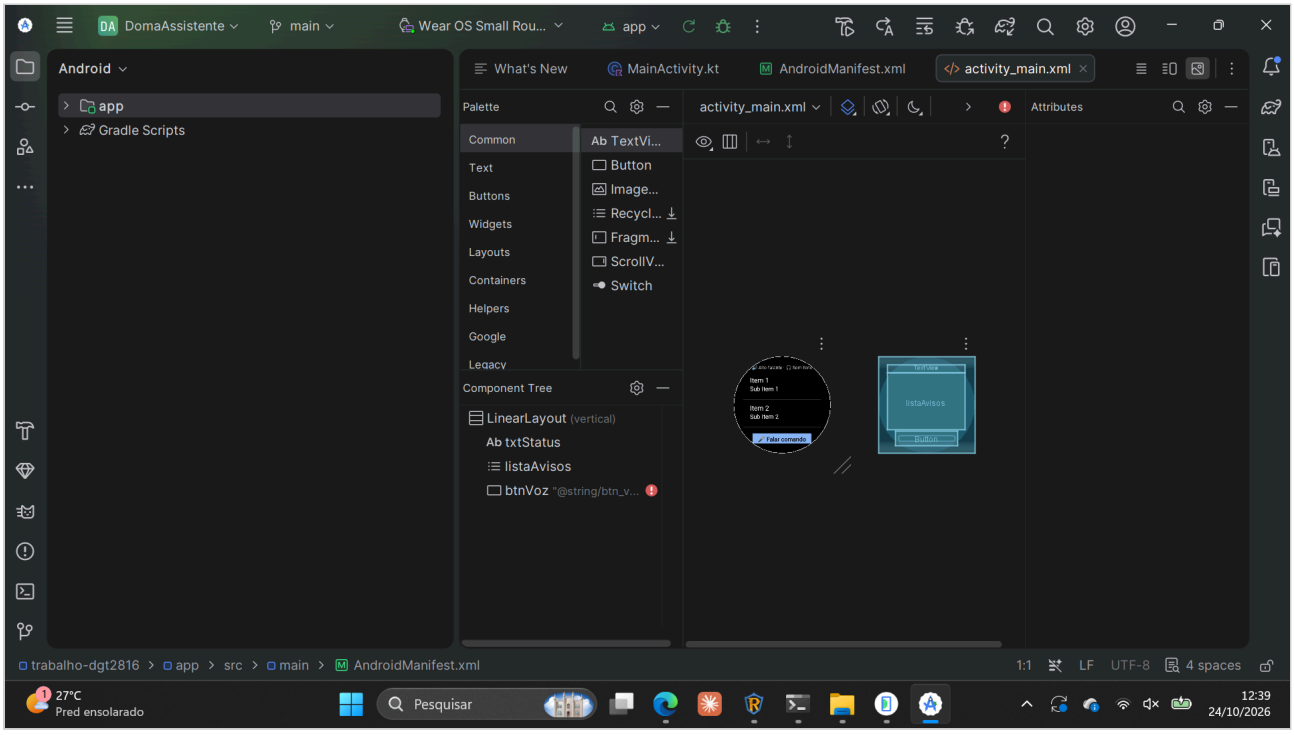


Figura 3 - Layout da tela no editor visual (txtStatus, listaAvisos e btnVoz).

Microatividade 4: criando o emulador

Criei o dispositivo virtual Wear OS Small Round com a imagem Wear OS 3 (API 30). Dados do emulador:

```
Name: Wear_OS_Small_Round
Device: wearos_small_round (Google)
Target: Wear OS 3 - Android 11.0 ("R"), android-wear/x86
```

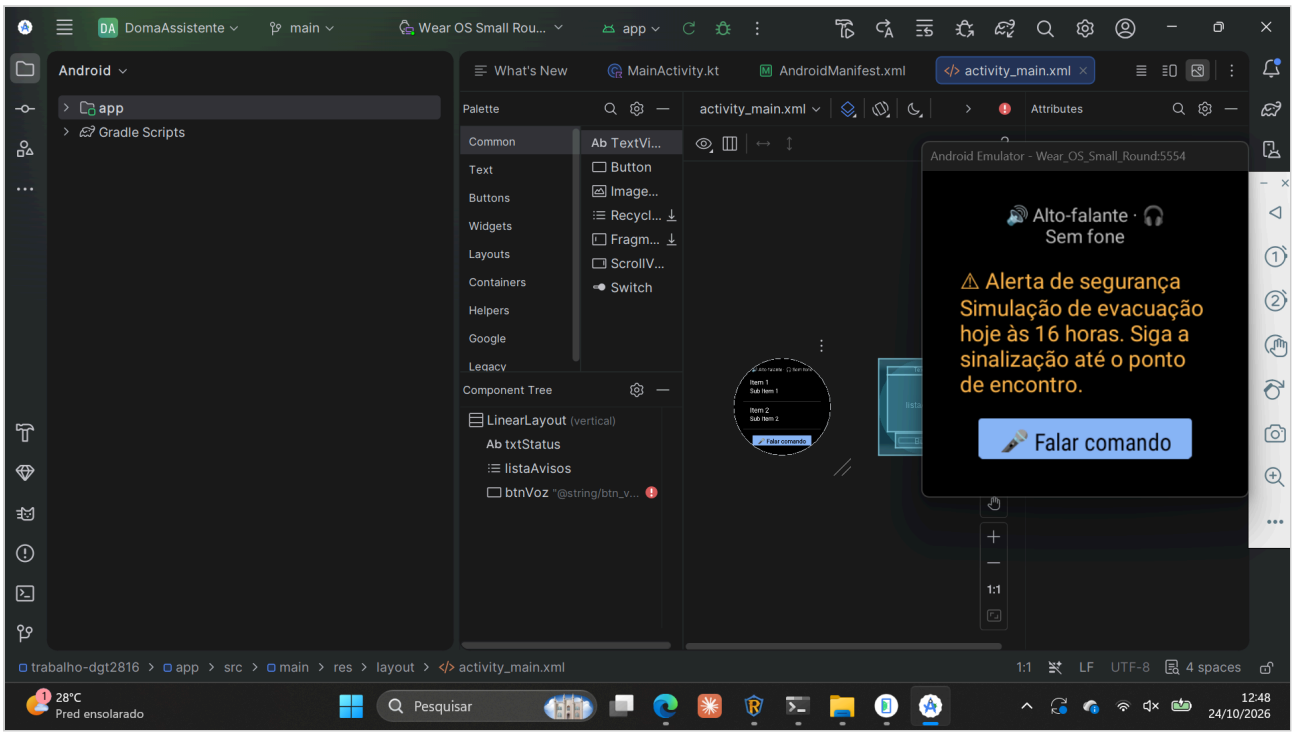


Figura 4 - O app rodando no emulador, na frente do Android Studio.

Microatividade 5: prints com o app complementar

Essa parte precisa de um relógio físico pareado com o celular, e eu não tenho um. Por isso tirei os prints do relógio direto do emulador. Com um relógio de verdade, o caminho seria ativar as opções do desenvolvedor no celular, abrir o app Wear OS, ir no menu de três pontos e tocar em "Fazer captura de tela do wearable".

3. O aplicativo



Figura 5 - Tela inicial. Em cima aparece se tem alto-falante e fone.

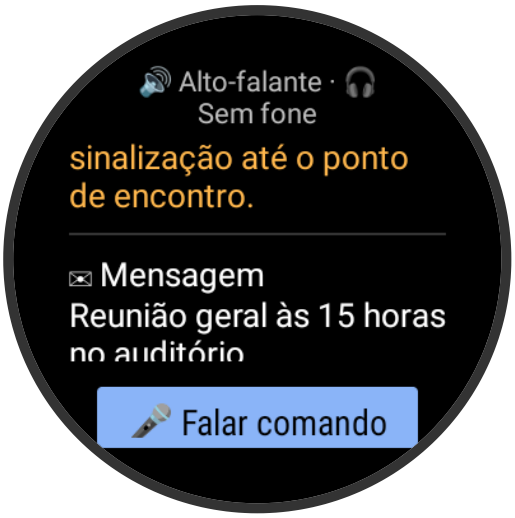


Figura 6 - Lista com mensagens e lembretes.

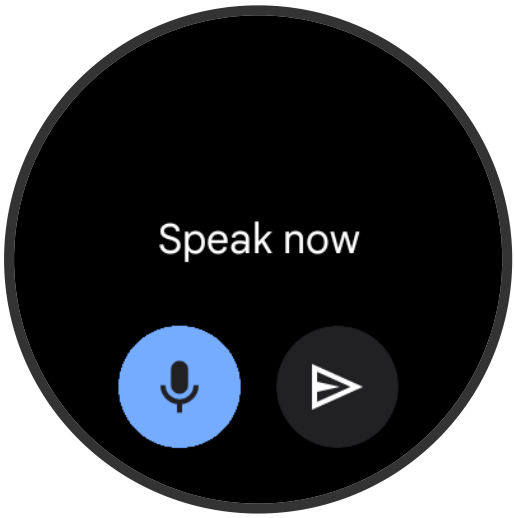


Figura 7 - Tela de voz que abre no botão "Falar comando".



Figura 8 - Lista de comandos.

Comandos que o app entende: "ler tudo", "ler mensagens", "ler alertas", "emergência", "conectar fone", "parar" e "ajuda". Antes de comparar, o texto falado é passado para minúsculas e sem acento, então "Emergência" e "emergencia" funcionam igual.

4. Como fiz cada passo do roteiro

Passo 2: saídas de áudio

Usei a função do roteiro quase igual, numa classe `AudioHelper`. Só precisei colocar `private val` no `context` do construtor, porque do jeito que estava no roteiro ele não compilava (o `context` não ficava acessível dentro da função).

```
fun audioOutputAvailable(type: Int): Boolean {
    if (!context.packageManager.hasSystemFeature(PackageManager.FEATURE_AUDIO_OUTPUT)) return
false
    return audioManager.getDevices(AudioManager.GET_DEVICES_OUTPUTS).any { it.type == type }
}
fun temAltoFalante() = audioOutputAvailable(AudioDeviceInfo.TYPE_BUILTIN_SPEAKER)
```

Passo 3: detectar quando o fone conecta

O callback é registrado no `onStart()` e removido no `onStop()`. Quando um fone conecta, o status do topo da tela muda e o relógio fala "Fone Bluetooth conectado".

```
override fun onAudioDevicesAdded(addedDevices: Array<out AudioDeviceInfo>?) {
    if (addedDevices.orEmpty().any { it.isSink && ehBluetooth(it.type) }) aoMudar(true)
}
```

Passo 4: abrir o Bluetooth

Se o relógio não tiver por onde tocar som, aparece uma janela "Sem saída de som" com o botão Conectar, que abre a tela de Bluetooth com a intent `ACTION_BLUETOOTH_SETTINGS` e os extras do roteiro.

Passos 5 e 6: tocar o áudio

Para falar usei o `TextToSpeech` em português. Nos alertas o texto vai pelo canal de alarme, assim ele é ouvido mesmo se o volume de mídia estiver baixo. O bipe é feito com `ToneGenerator` e a vibração com `VibrationEffect`.

```
if (alerta) params.putInt(TextToSpeech.Engine.KEY_PARAM_STREAM, AudioManager.STREAM_ALARM)
tts.speak(texto, modo, params, id)
```

5. Testes no emulador

- O app compilou e instalou no emulador sem erro.
- O status mostrou "Alto-falante · Sem fone", que bate com o que o emulador informa (`dumpsys audio` lista só o speaker).
- Ao abrir, o relógio fala a mensagem de boas-vindas.
- Tocando num alerta de segurança: bipe, vibração e a fala saindo pelo canal de alarme (conferi no `dumpsys audio`, aparece `USAGE_ALARM`).
- O botão "Falar comando" abre a tela de voz do Wear OS. Cancelando, aparece a lista de comandos.
- O comando "Emergência" toca o alarme e fala a instrução de evacuação.

6. Dificuldades que tive

- **Versão do Java.** O Android Studio novo vem com Java 25, e a primeira versão do Gradle que coloquei no projeto não aceitava. Tive que atualizar para o Android Gradle Plugin 9.4 e Gradle 9.8. Nessa versão o Kotlin já vem embutido no plugin do Android, então tirei o plugin separado do Kotlin.
- **Tela redonda.** Na primeira versão o texto do topo e os cantos do botão ficavam cortados pela tela redonda. Aumentei as margens e diminuí um pouco as letras.
- **Voz no emulador.** O emulador não tem microfone configurado, então a tela de voz não reconhecia nada e o app ficava parado. Fiz o app mostrar a lista de comandos quando não vem nenhuma frase. No começo a lista abria e fechava sozinha. Descobri que o mesmo "voltar" que fechava a tela de voz também fechava a lista, e resolvi com um pequeno atraso.

7. Conclusão

Com esse trabalho aprendi a montar o ambiente do Android Studio para Wear OS e a trabalhar com as saídas de áudio do relógio: verificar se tem alto-falante ou fone, perceber quando o fone conecta e mandar o usuário para o Bluetooth quando precisa. O app mostra que um relógio pode ajudar quem tem deficiência visual a receber mensagens e alertas importantes sem precisar olhar para a tela.